Documentation de validation

Équipe GL36

Ensimag - Projet Génie Logiciel

22 janvier 2026

Résumé

Ce document présente la stratégie de validation du compilateur `decac`. Il détaille l'articulation (à l'aide de `Maven`) entre les tests unitaires sous JUnit/Mockito, les tests fonctionnels `.deca` et l'infrastructure d'automatisation des scripts shell.

Table des matières

1 Stratégie de validation **3**

1.1 Types de tests par étape 3

1.1.1 Phase Lexicographique et Syntaxique (Étape A) 3

1.1.2 Phase Contextuelle (Étape B) 3

1.1.3 Génération de Code (Étape C) 4

1.2 Organisation des tests 4

1.2.1 Organisation des fichiers Deca (`src/test/deca/`) 5

1.2.2 Organisation des tests Java (`src/test/java/`) 5

1.3 Objectifs et atteintes 6

2 Infrastructure de test **6**

2.1 Intégration sous Maven 6

2.2 Détail des scripts d'automatisation 7

2.2.1 Analyse lexicale (`test-lex.sh`) 7

2.2.2 Validation de la syntaxe et indempotence (`test-synt.sh`) 7

2.2.3 Analyse Contextuelle et Vérification des Entêtes (`test-context.sh`) . . 7

2.3 Validation des Options du Compilateur (`test-decac.sh`) 7

3 Gestion des risques et des rendus **7**

3.1 Analyse des dangers critiques 8

3.2 Gestion des rendus et calendrier 9

3.3 Procédure de validation de rendu 9

4 Résultats de couverture (Jacoco) **10**

4.1 Résultats de la couverture 10

4.2 Analyse des résultats 10

1 Stratégie de validation

La validation du compilateur Deca est effectuée à plusieurs niveaux. Nous distinguons la validation interne des structures de données (tests unitaires) de la validation du comportement global du compilateur vis-à-vis du langage Deca (tests fonctionnels).

1.1 Types de tests par étape

La quantité et la nature des tests varient selon l'étape de compilation visée.

1.1.1 Phase Lexicographique et Syntaxique (Étape A)

L'objectif est de vérifier la robustesse de nos règles syntaxiques (Lexer/Parser). Étant donné que la majorité des règles lexicales était fournie, les tests fonctionnels ont été principalement destinés à augmenter la couverture avec des cas valides (mots-clés Deca, accents et symboles, etc.) ainsi qu'à cibler les aspects délicats via des tests invalides (identificateurs mal formés, commentaires non terminés, etc.). Le launcher `test_lex` a été utilisé systématiquement pour déboguer le lexer et identifier précisément les tokens reconnus.

Pour la validation du parser, deux approches ont été privilégiées. D'une part, les tests nécessitant une vérification de la structure de l'arbre abstrait produit (priorité des opérateurs, dépassement de capacité des entiers ou des flottants) ont été validés manuellement en inspectant l'arbre généré au moyen du launcher `test_synt`. D'autre part, le reste des tests a été validé à l'aide d'un script automatique s'appuyant sur le théorème de la décompilation : pour chaque fichier source, nous vérifions que le code décompilé compile à nouveau sans erreur vers un arbre abstrait identique.

1.1.2 Phase Contextuelle (Étape B)

Pour cette étape, nous avons mis en place deux stratégies complémentaires :

- **Tests unitaires (JUnit & Mockito)** : Afin de cibler précisément chaque règle contextuelle, nous avons développé une suite de tests unitaires. Pour chaque classe `<Nom_de_la_classe>` du package `tree` soumise aux tests, il existe une classe associée `Test<Nom_de_la_classe>` regroupant les vérifications sémantiques (positives ou négatives). Dans une optique de développement piloté par les tests (TDD), cette approche permet d'implémenter les méthodes de vérification de manière isolée. Le comportement des dépendances est simulé à l'aide de `Mockito`, permettant de créer des « mocks » et de définir leurs comportements attendus sans nécessiter l'implémentation complète des classes liées. Notre stratégie reposait fortement sur les tests unitaires ceux-ci sont parfois assez avancés. La classe de test `TestPlusAdvanced` initalement fournie a par exemple été refactorisée dans une classe `TestOpArith` qui passe la suite de tests pour les 4 opérateurs arithmétiques binaires.
- **Tests fonctionnels** : Une suite de fichiers `.deca`, classés par catégories (classes, expressions, contrôle), a été constituée a posteriori. Elle permet d'améliorer la couverture globale et de s'assurer que la chaîne de vérification contextuelle fonctionne correctement dans son ensemble, au-delà de l'unité des nœuds.

Détails des tests unitaires avancés La validation contextuelle repose sur des tests unitaires avancés visant à garantir la robustesse sans duplication de code. La classe `TestOpArith`

illustre cette volonté d'efficacité : initialement limitée à l'opérateur `Plus`, elle a été refactorisée pour valider simultanément les quatre opérateurs arithmétiques binaires (`Plus`, `Minus`, `Multiply`, `Divide`). En utilisant des `@ParameterizedTest` [1] et une `@MethodSource`, nous passons les constructeurs des nœuds via l'interface `BiFunction` [2]. Chaque itération reçoit un objet `Arguments` [3] contenant la référence du constructeur (ex : `Plus::new`), assurant une validation uniforme entre les opérateurs tout en minimisant la redondance de code.

D'autres tests nécessitent de simuler des interactions fines avec les structures internes du compilateur :

- Dans la classe `TestInitialization`, nous avons été confrontés à la nécessité de tester la méthode `verifyRValue`. L'utilisation d'un simple *mock* pour l'expression à vérifier entraînait des erreurs de type `NullPointerException`. Nous avons donc implémenté les tests avec des **Spies** [4]. Cela nous a permis de conserver le comportement réel du nœud tout en redéfinissant sélectivement certains retours via `doReturn(...).when(...)` [5].
- Dans la classe `DeclVar`, l'objet `VariableDefinition` est instancié à l'intérieur de la méthode `verifyDeclVar`, il est impossible d'y accéder directement pour vérifier si le type ou le symbole enregistré est correct. Nous avons résolu cette problématique en utilisant un `ArgumentCaptor` [6]. Cet outil nous permet d'intercepter l'objet au moment où il est passé à la méthode `env.declare()` et d'en inspecter les attributs a posteriori. Enfin, pour tester la robustesse face aux erreurs de double déclaration, nous avons utilisé `doThrow(...).when(...)` [7] sur des méthodes `void`, simulant ainsi une remontée d'exception depuis les couches inférieures de l'environnement vers le nœud de déclaration.

1.1.3 Génération de Code (Étape C)

La validation repose sur des tests fonctionnels exécutés par la machine cible. Le compilateur produit un fichier `.ass` qui est ensuite interprété par `ima`. Trois issues sont possibles :

1. La machine s'arrête brutalement (erreur dans le code généré, instruction mal utilisée, conflit mémoire).
2. La machine lève une erreur à l'exécution gérée par le `RuntimeErrorManager` (comportement attendu pour les tests d'invalidité dynamique comme la division par zéro).
3. Le programme s'exécute correctement et produit le résultat attendu.

1.2 Organisation des tests

Chaque test fonctionnel contient un en-tête fournissant une description, le résultat attendu, ainsi que sa date de création.

Exemple d'en-tête d'un test contextuel

```
// Description:
//    Test de déclaration de méthode invalide : paramètres de même nom.
//    Le paramètre 'a' est déclaré plusieurs fois.
//
// Resultats:
//    Erreur contextuelle
//    Ligne 13 : Parameter a is already defined
//
```

```
// Historique:
//    cree le 18/01/2026
```

Exemple d'en-tête d'un test de génération de code

```
// Description:
//    Test de débordement du tas.
//    On utilise un objet avec beaucoup de champs pour épuiser le tas
//    rapidement avant que la pile ne déborde par récursion.
//
// Resultats:
//    Error: Heap overflow
//
// Historique:
//    cree le 19/01/2026
```

NB : À noter que les scripts de test automatique pour la génération de code et l'analyse contextuelle s'appuient sur ces en-têtes pour effectuer les vérifications et comparer les résultats obtenus aux résultats attendus. Il est donc impératif d'en respecter le format.

1.2.1 Organisation des fichiers Deca (`src/test/deca/`)

Ce répertoire reflète l'architecture du compilateur. Chaque sous-répertoire contient les dossiers `/valid`, `/invalid` et `/provided` (ainsi que `/perf`, `/extension` ou `/interactive` selon le besoin). En complément de ce squelette, chaque dossier est organisé thématiquement pour faciliter la navigation dans la base de tests. Les fichiers sont nommés de manière explicite pour indiquer la fonctionnalité testée.

- **codegen/ :** Tests pour l'étape C.
  - `valid/` : Tests structurés par thématiques (`arithmetic/`, `boolean/`, `class/`, etc.) vérifiant la correction de l'exécution.
  - `invalid/` : Focus sur les erreurs à l'exécution (`null-dereference`, `stack-overflow`).
  - `interactive/` : Tests nécessitant des entrées utilisateur
- **context/ :** Tests pour l'étape B. Séparés en `valid/` pour les succès de typage et `invalid/` pour les erreurs sémantiques attendues (ex : `assign/float-to-int.deca`).
- **syntax/ :** Tests pour l'étape A (Lexer, Parser, et gestion des directives `include`).

1.2.2 Organisation des tests Java (`src/test/java/`)

Les tests unitaires sont regroupés par package technique :

- `fr.ensimag.deca.context` : Tests de bas niveau sur la table des symboles et les environnements (`TestEnvironmentExp.java`).
- `fr.ensimag.deca.tree` : Tests des nœuds de l'arbre abstrait (`TestAssign.java`, `TestWhile.java`, etc.). Chaque fichier JUnit correspond à une classe spécifique de l'AST.

1.3 Objectifs et atteintes

L'objectif principal de notre stratégie de validation est d'assurer une couverture maximale des fonctionnalités du compilateur Deca, tout en garantissant la robustesse et la fiabilité du code généré. Il n'y a pas grand intérêt à tester seulement la couverture, car un code qui parcourt une bonne partie des lignes peut très bien ne pas détecter des erreurs critiques. La stratégie était plutot de faire des tests tres ciblés, en réfléchissant aux cas limites et aux scénarios d'erreurs probables et en écrivant des tests pour ceux-ci.

Les résultats obtenus sont plutôt satisfaisant mis à part la partie géneration de code du langage essentiel (la gestion des méthodes et leur appel) qui n'a pas été suffisamment testée et comporte des bugs souvent inconnus.

2 Infrastructure de test

L'automatisation est au cœur de notre stratégie de validation, pour assurer une intégration continue efficace et reproductible. Nous avons mis en place, à l'aide de la configuration Maven fournie (légèrement modifiée), la gestion d'une suite de scripts shell automatiques.

2.1 Intégration sous Maven

Le fichier `pom.xml` centralise la configuration de notre environnement de test. Nous utilisons principalement deux plugins pour piloter la validation :

- Maven Surefire Plugin : Gère l'exécution des tests unitaires JUnit
- Exec Maven Plugin : Permet de lancer tous nos scripts shell durant la phase `test`
- JaCoCo Maven Plugin : Intégré pour mesurer la couverture de code lors de l'exécution des tests

Commandes utiles
Pour lancer les tests unitaires Java uniquement il suffit de faire :

```
$ mvn surefire:test
```

Pour lancer un script spécifique il suffit d'exécuter, en s'assurant de l'avoir rendu exécutable (`chmod +x`) :

```
$ ./src/test/script/<nom_du_script>.sh
```

Pour lancer la suite complète de tests (unitaires + scripts shell) il suffit d'exécuter :

```
$ mvn test
```

NB : `mvn test -Dskip.provided.tests=true` permet de se concentrer uniquement sur nos tests en ignorant les scripts fournis initialement (utilisé lors du développement du sans objet).

2.2 Détail des scripts d'automatisation

Chaque script situé dans `src/test/script/` couvre une étape spécifique du processus de compilation.

2.2.1 Analyse lexicale (`test-lex.sh`)

Ce script valide la première étape de compilation. Pour les fichiers valides, il s'assure de l'absence de message d'erreur. Pour les fichiers invalides (identificateurs mal formés, etc.), il vérifie que le `lexer` lève bien une erreur. *NB : Ce script ne doit être utilisé qu'après avoir manuellement validé que les tokens reconnus sont corrects avec le launcher* ***`test_lex`****.*

2.2.2 Validation de la syntaxe et indempotence (`test-synt.sh`)

Ce script implémente un test de cohérence avancé basé sur le théorème de décompilation.

1. Il compile un fichier source vers un premier arbre (`AST_ORIGINAL`).
2. Il décompile cet arbre en code source Deca via l'option `-p` de `decac`.
3. Il re-compile ce code généré pour produire un second arbre (`AST_DECOMPILED`).

Le script utilise `sed` pour ignorer les informations de positionnement `[LIGNE, COLONNE]` et compare les deux structures via `diff`. Si les structures d'arbres sont identiques, le test est considéré comme réussi.

2.2.3 Analyse Contextuelle et Vérification des Entêtes (`test-context.sh`)

Ce script automatise la vérification sémantique (Étape B). Pour les tests invalides, il extrait avec `grep` et `sed` l'erreur attendue (ligne et message) directement depuis l'entête du fichier `.deca`. Il compare ensuite ces attentes avec la sortie réelle du compilateur (`decac -v`), garantissant que l'erreur levée est exactement celle prévue.

NB : Il est donc impératif de respecter le format des en-têtes **`??`** *dans chaque fichier de test, auquel cas les tests ne passseront pas correctement.*

2.3 Validation des Options du Compilateur (`test-decac.sh`)

Ce script teste la conformité de l'interface en ligne de commande (CLI). Il vérifie notamment chaque option, notamment :

- Gestion des registres (`-r`) : Validation du comportement pour les limites acceptées (4-16) et rejet des valeurs invalides.
- Suppression des vérifications (`-n`) : Le script vérifie par un `grep` sur le fichier `.ass` que les instructions de branchement vers `ERROR` (division par zéro, stack overflow) ont bien été retirées, puis confirme l'absence de plantage par une exécution réelle dans `ima`.

3 Gestion des risques et des rendus

Afin d'évaluer et de prévenir au mieux les risques du projet, nous avons identifié les dangers susceptibles d'affecter la qualité ou le bon fonctionnement du projet. Ces risques ont été classés par ordre décroissant selon leur rapport coût-bénéfice (du plus critique au moins critique), conformément aux recommandations des enseignants.

3.1 Analyse des dangers critiques

Pour chaque danger identifié, nous avons défini une ou plusieurs actions facilement vérifiables, permettant d'éviter ce risque ou d'en limiter l'impact. Ces dangers sont, à ce stade, centrés sur le sous-langage «sans objet», mais s'appliquent également à la version complète du langage ainsi qu'au développement de l'extension. Une réévaluation des risques sera donc réalisée après le rendu intermédiaire.

- **Validité fonctionnelle** : Pour le rendu intermédiaire, il est impératif que l'intégralité de la chaîne de compilation du sous-langage «sans objet» soit fonctionnelle (conception, vérifications contextuelles et génération de code).
  ⇒ À partir de l'exemple fourni dans le polycopié, nous avons identifié les points critiques, notamment l'ajout de la classe `ConvFloat` à l'arbre abstrait lors de certaines opérations arithmétiques ou d'assignations, ou encore l'appel à `ReadInt`. Ces points ont été ajoutés aux tests unitaires de l'étape d'analyse contextuelle en plus des règles données par la spécification du langage Deca.
- **Conformité de l'interface et automatisation** : Le non-respect des spécifications de la commande `decac` (options `-p`, `-v`, `-r` etc.) ou une sortie interactive lors de `mvn test` rendrait le compilateur incompatible avec les scripts d'évaluation automatique des enseignants.
  ⇒ Nous avons mis en place un script de test d'interface dédié vérifiant le code de retour (*exit status* 0 ou 1) ainsi que le format de sortie pour chaque option. Les tests exécutés automatiquement par `Maven` sont strictement non-interactifs. Les programmes nécessitant une interaction utilisateur (`readInt`, `readFloat` par exemple) ou les scripts utilisés lors des démos en séance de suivi, sont isolés dans des sous-répertoires `/interactive` et exclus de l'exécution automatique.
- **Validité des oracles** : Il existe un risque que nos tests passent alors que le compilateur produit des résultats incorrects (acceptation de programmes invalides, rejet de programmes valides, AST incohérent ou exceptions Java non-rattrapées).
  ⇒ Pour y pallier nous utilisons des oracles automatiques, c'est-à-dire des critères qui déterminent automatiquement si un test a réussi ou échoué. Par exemple la vérification de l'idempotence de la décompilation, la cohérence des numéros de lignes d'erreur ou encore l'absence d'exceptions Java non-rattrapées. Ces oracles sont inspirés des exemples fournis dans le polycopié.
- **Erreurs de régression** : Le développement de la génération de code pourrait, à travers certaines erreurs, rendre le compilateur invalide sur des étapes précédentes pourtant fonctionnelles.
  ⇒ Intégration du script `common-tests.sh` dans notre cycle de développement. L'exécution de la commande `mvn clean test` doit être réalisée avant chaque *push* sur le dépôt afin de garantir que les nouvelles fonctionnalités ne cassent pas les fonctionnalités déjà implémentées.
- **Risque de synchronisation Git** : L'oubli d'un fichier source ou l'ajout accidentel de fichiers binaires/auxiliaires (`.class`, etc.) pourrait créer des conflits ou empêcher la compilation correcte sur une autre machine.
  ⇒ Nous avons séparé les fichiers générés dans des répertoires spécifiques (`target/` et `/docs/build`) pour minimiser les erreurs d'oubli dans le `.gitignore`. Avant chaque rendu, un `git clone` est réalisé dans un répertoire vierge pour vérifier que la compilation (`mvn test-compile`) et les tests (`mvn test`) fonctionnent sans dépendances locales (cf. 3.3)

3.2 Gestion des rendus et calendrier

La réussite du projet repose aussi sur la gestion du calendrier et des livrables. Nous avons identifié deux dangers organisationnels majeurs : le dépassement des échéances et la livraison d'une version instable du projet.

- **Respect des échéances** : L'oubli d'une date de rendu, de suivi, de soutenance ainsi qu'un des documents qui nous sont demandés pour ces écheances n'est pas souhaitable.
  ⇒ Nous avons ajouté à notre planning des jalons anticipés, fixés 24 heures avant chaque rendu officiel, passé lesquels aucune nouvelle fonctionnalité ne doit être ajoutée. Ces deux jours sont réservés exclusivement à la vérification et la validation de rendu. Le planning partagé récapitule les dates clés du projet ainsi que les jalons internes.
- **Gestion des versions** : Le danger est de livrer une version instable suite à l'intégration de dernière minute d'une fonctionnalité complexe.
  ⇒ Pour le rendu intermédiaire, la priorité est donnée au compilateur «sans objet» sur la branche `main`. Toute autre fonctionnalité (extensions, partie objet) est développée sur des branches séparées et ne sera fusionnée qu'après validation.

3.3 Procédure de validation de rendu

Pour reproduire notre validation, voici la liste des tâches exécutées avant chaque rendu.

- ☐ Lancer `mvn test` pour exécuter l'ensemble des tests (unitaires et fonctionnels). Cette commande exécute aussi le script `common-tests.sh` : aucune erreur ne doit survenir.
- ☐ Vérifier que les tests interactifs (utilisant `readInt` ou `readFloat`) sont tous situés dans des sous-répertoires `/interactive`.
- ☐ Vérifier le format des en-têtes `// Description` et `// Resultats` de chaque fichier de test `.deca` et s'assurer que la ligne d'erreur attendue pour les tests `invalid` correspond à la sortie du compilateur.
- ☐ Exécuter la procédure de simulation de rendu ci-dessous.

Simulation de rendu : L'usage du répertoire `/tmp/` garantit que ce dernier sera nettoyé au redémarrage de l'ordinateur et évite ainsi de polluer les dossiers de travail personnels avec des versions dupliquées du projet, mais le choix d'un autre dossier de destination n'importe pas.

1. Clonage du dépôt dans un répertoire temporaire :

```
$ git clone https://gitlab.ensimag.fr/gl2026/gr6/gl36.git
     /tmp/Projet_GL_Rendu
  cd /tmp/Projet_GL_Rendu
```

2. Vérification de la compilation et des tests :

```
$ mvn test
```

3. Vérification de l'exécutable :

```
$ ./bin/decac -p src/test/deca/syntax/valid/hello.deca
```

4 Résultats de couverture (Jacoco)

Comme mentionné précédemment, nous utilisons **JaCoCo** pour mesurer l'efficacité de la couverture de notre suite de tests.

4.1 Résultats de la couverture

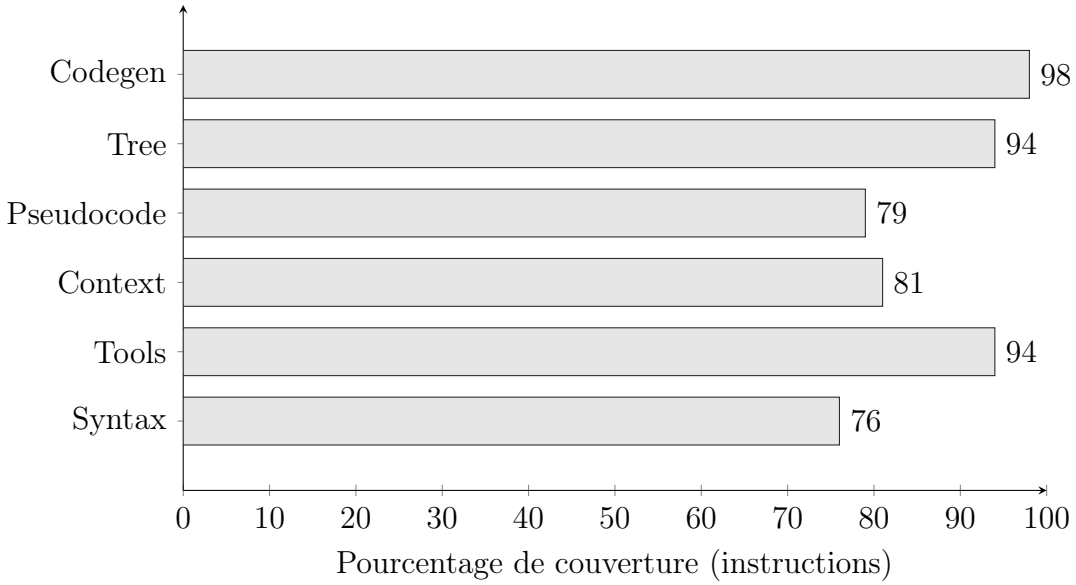


FIGURE 1 – Couverture JaCoCo par paquetage (en pourcentage d'instructions)

NB : Les rapports détaillés sont générés dans `target/site/jacoco/index.html` *après chaque exécution de* `mvn verify -Djacoco.skip=false`.

4.2 Analyse des résultats

Fiabilité de la couverture
Paradoxalement, le package `codegen` affiche la meilleure couverture (98%), alors que c'est la phase ou nous avons rencontré le plus d'erreurs après le rendu du projet. Cela s'explique notamment par le fait que ce package ne contient que les `Managers` et montre ainsi qu'il n'y a pas de méthodes ou de champs non utilisés même si cela ne garantit pas que la logique interne fonctionne correctement.

Rigueur de l'analyse contextuelle
Le package `context` (81%) et les outils associés (`tools` à 94%) démontrent que notre stratégie de tests unitaires et fonctionnels a réussi, couvrant la quasi-totalité des vérifications de types et de la gestion de la table des symboles.

Zones d'amélioration
Le package `syntax` (76%) affiche un score plus bas, ce qui est structurel : une partie du code est générée par ANTLR et correspond à des chemins de récupération d'erreurs syntaxiques complexes qu'il est difficile de provoquer intégralement. De même, malgré les efforts, les scores

correspondant aux "branches manquées" sont plus faible ce qui indique que nous aurions pu tester d'avantage tous les cas qu'ils soient positifs ou négatifs.

En définitive, le compilateur atteint une couverture globale de 83% des instructions couplée à une validation de 71% des branches logiques ce qui témoigne d'une stratégie de test rigoureuse et fiable.